

CMSC828G: Special Topics in Systems for Machine Learning

Introduction to HPC

Instructor: Yafan Huang



DEPARTMENT OF
COMPUTER SCIENCE

Introduce Yourself

- Name
 - PhD/MS/Undergraduate Student
 - Why do you expect from this course?
 - Something interesting about yourself
-
- TA: Shweta Bhardwaj (Email: shweta12@umd.edu)
 - TA Office Hour: IRB 3119, Friday, 12:30 pm to 2:30 pm.

Getting Start with Zaratan

ssh username@login.zaratan.umd.edu

- Zaratan is managed by UMD DIT.
- 360 compute nodes each with 128 CPU cores and 512 GiB of memory.
- 20 nodes with 4 NVIDIA A100 GPUs (40 GB per GPU).
- 8 nodes with 4 NVIDIA H100 GPUs (80 GB per GPU).

Details of the Zaratan cluster: [1]

Description	Number of nodes	Processor	Cores/node	Mem/node (GiB)	Mem/core	/tmp size (TB)	GPUs/node
Standard compute	360	AMD Zen3	128	512	4	0.75	none
Large memory	6	AMD Zen3	128	2048	16	6	none
H100	8	Intel SapphireRapids	96	512	5.3	12	4 x H100 (80 GB)
A100	20	AMD Zen3	128	512	4	0.75	4 x A100

[2]

Accelerate llama.cpp on AMD EPYC: ZenDNN Delivers Up to 4.5x the Prompt Processing

📅 Aug 18, 2026

[1] <https://hpcc.umd.edu/home/clusters/zaratan/>

[2] <https://www.amd.com/en/developer/resources/technical-articles/2026/llama-cpp-on-amd-epyc.html>

Getting Start with Nexus (nexusclass)

- Nexus is managed by UMD CS/UMIACS.
 - *nexusclass* partition.
- 39 nodes with 4 NVIDIA A4000 GPUs.
- 16 nodes with 8 NVIDIA A5000 GPUs.

`ssh username@nexusclass.umiacs.umd.edu`

GPU Features	NVIDIA RTX A4000
GPU Memory	16GB GDDR6 with error-correction code (ECC)
Display Ports	4x DisplayPort 1.4
Max Power Consumption	140 W
Graphics Bus	PCI Express Gen 4 x 16
Form Factor	4.4" (H) x 9.5" (L) Single Slot
Thermal	Active
VR Ready	Yes

[1]

GPU Features	NVIDIA RTX A5000
GPU Memory	24GB GDDR6 with error-correction code (ECC)
Display Ports	4x DisplayPort 1.4*
Max Power Consumption	230 W
Graphics Bus	PCI Express Gen 4 x 16
Form Factor	4.4" (H) x 10.5" (L) Dual Slot
Thermal	Active
NVLink	2-way low profile (2-slot and 3-slot bridges)
Virtual GPU (vGPU) Software Support	NVIDIA Virtual PC (vPC) and Virtual Applications (vApps), NVIDIA RTX vWS, NVIDIA Virtual Compute Server
vGPU Profiles Supported	See the Virtual GPU Licensing Guide
VR Ready	Yes

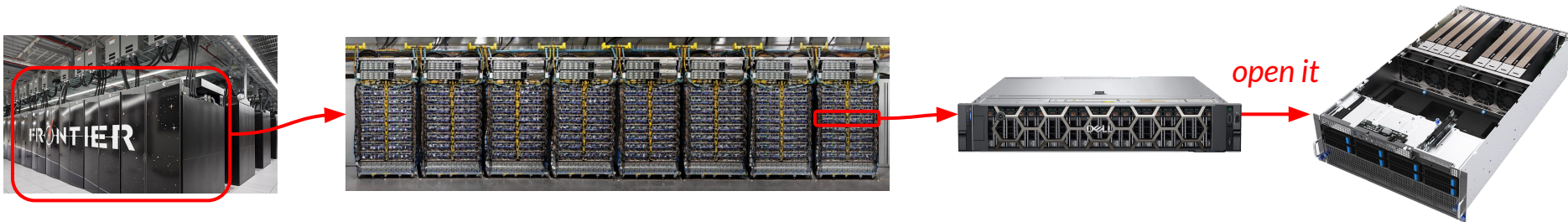
[2]

[1] <https://www.nvidia.com/en-us/products/workstations/rtx-a4000/>

[2] <https://www.nvidia.com/en-us/products/workstations/rtx-a5000/>

High-Performance Computing

- High-performance computing (HPC) is the use of supercomputers and computer clusters to solve advanced problems. – *Definition from Wikipedia.*

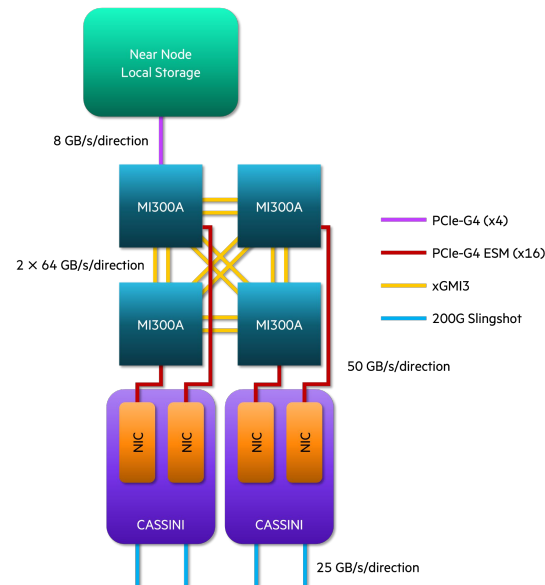


- Non-Uniform Memory Access (NUMA): a memory architecture where memory access time depends on the memory location relative to the processor.
- Most modern HPC systems are organized in this hierarchical way.

Top500 Supercomputer List

[1]

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	LineShine - LingKun, LX2 304C 1.55GHz, LingQi, Kylin OS, Shenzhen Cloud Computing Center Co., Ltd. National Supercomputing Centre in Shenzhen (NSCS) China	13,789,440	2,198.40	2,735.82	42,220
2	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,340,000	1,809.00	2,821.10	29,685
3	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
4	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
5	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, Bull EuroHPC/FZJ Germany	4,801,344	1,000.00	1,226.28	15,794



Connection topology in one node of El Capitan^[2]

[1] <https://top500.org/lists/top500/2026/06/>

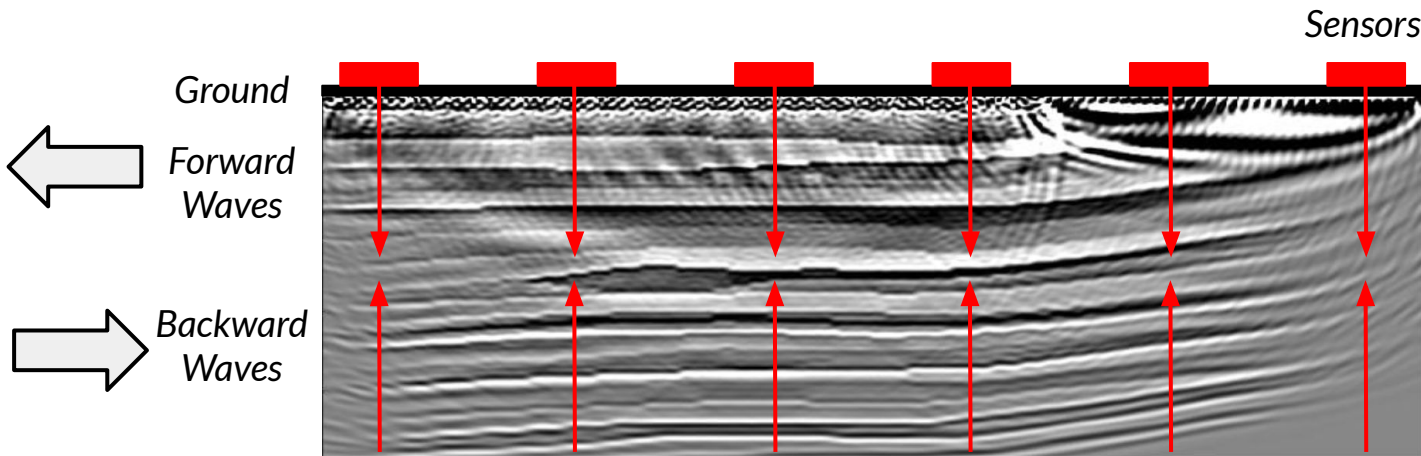
[2] <https://hpc.llnl.gov/documentation/user-guides/using-el-capitan-systems/hardware-overview>

Case 1: Seismic Imaging

- Reverse time migration (RTM) records seismic waves to create a high-resolution image of the Earth's subsurface. – *Used a lot for finding oils or water resources.*
- RTM is highly computation-intensive, with each step designed to run in parallel.
- RTM is I/O-intensive, thousands of wave snapshots are sent into disk.



Supercomputer

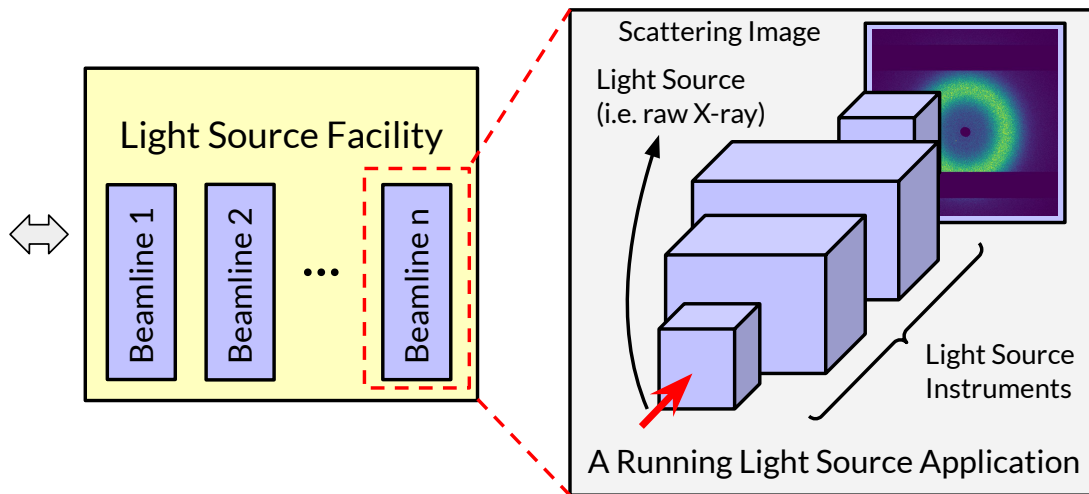


Case 2: Light Source

- Light source applications produce intense and tunable X-rays, enabling the study of both dynamic and static features of materials.
- Data generation speed for light source facility can be extremely high, requiring computation/storage from nearby supercomputers.



*Light Source Facility at
Argonne National Laboratory, IL*



Case 3: Quantum Simulation

- Quantum simulation demands significant storage and computational resources.
- Storage constraints: only 48-qubit quantum circuit can fully occupy Frontier memory systems (4.6 Peta Bytes of DDR4 memory).
- Computation constraints: every small step in quantum simulation (a.k.a. state-vector simulation) is a large-scale vector matrix multiplication.



Frontier Supercomputer

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

Example of Vector
Matrix Multiplication

Case 4: LLM Training

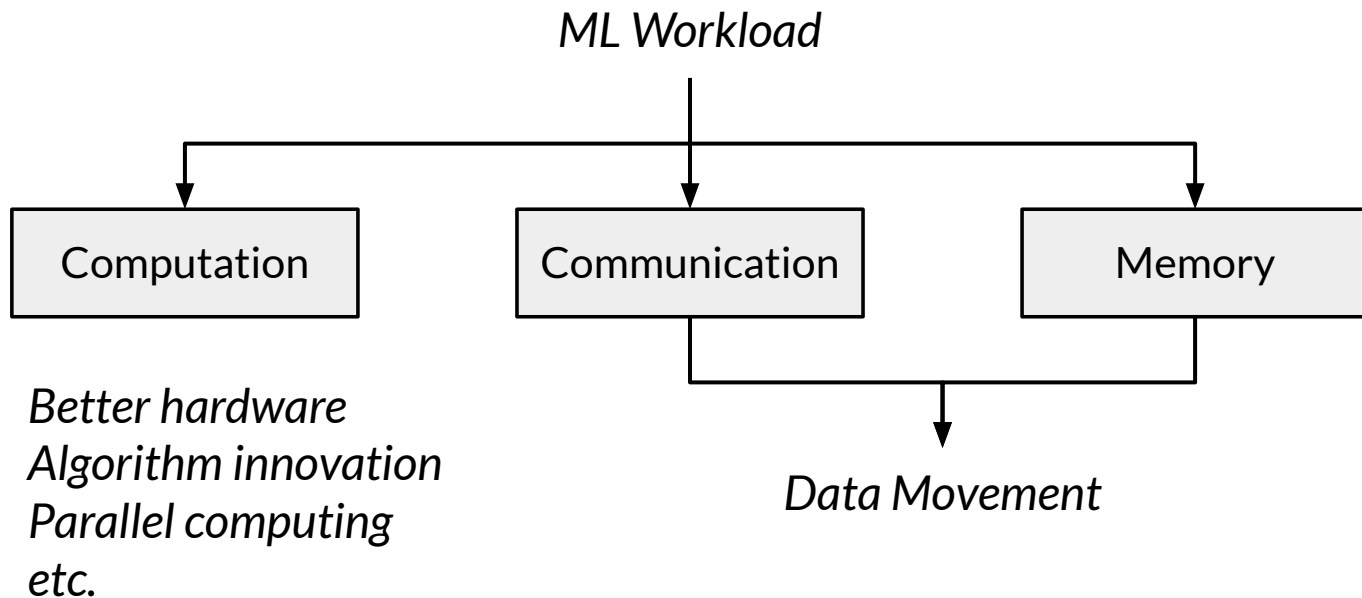
- LLM training has become a major HPC workload.
- GPT-4 Training Efforts: 25,000 GPUs, 90~100 Days.



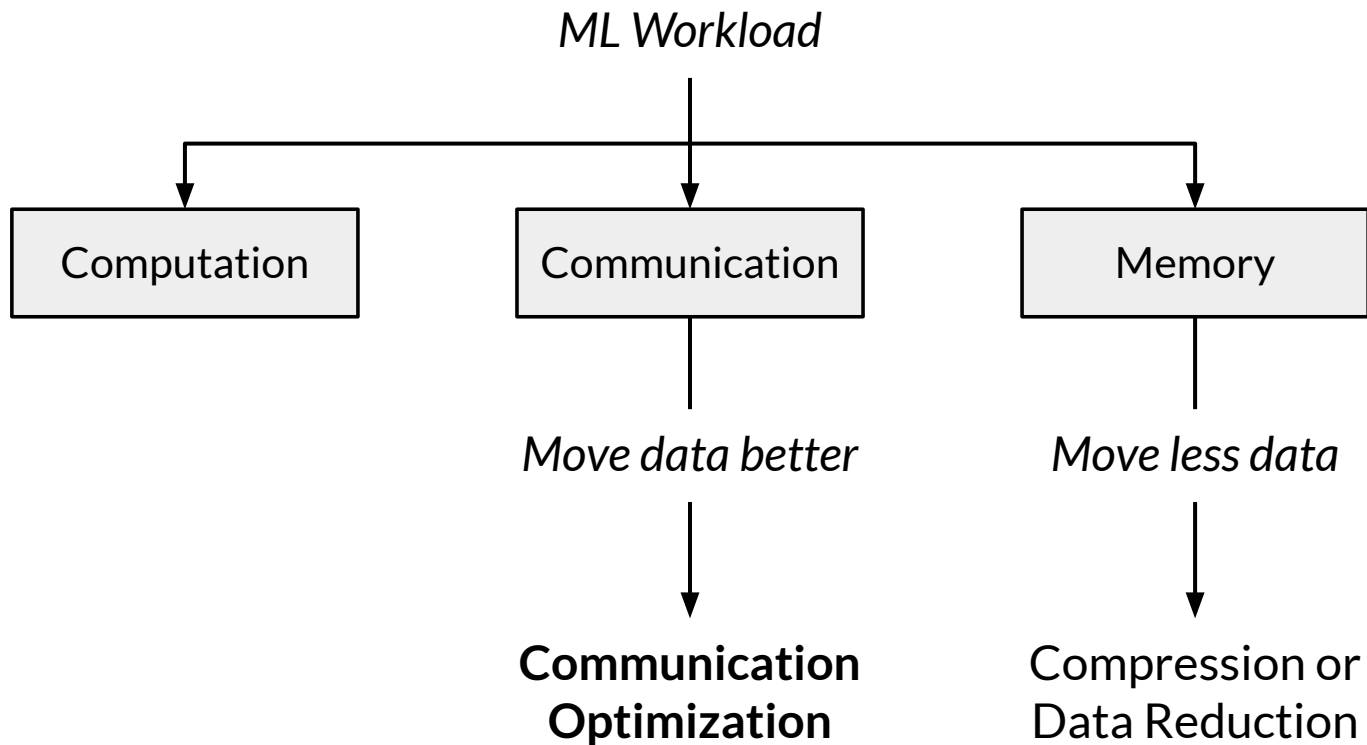
- What about LLM inference? Is it HPC?



From HPC to ML Systems: A System View

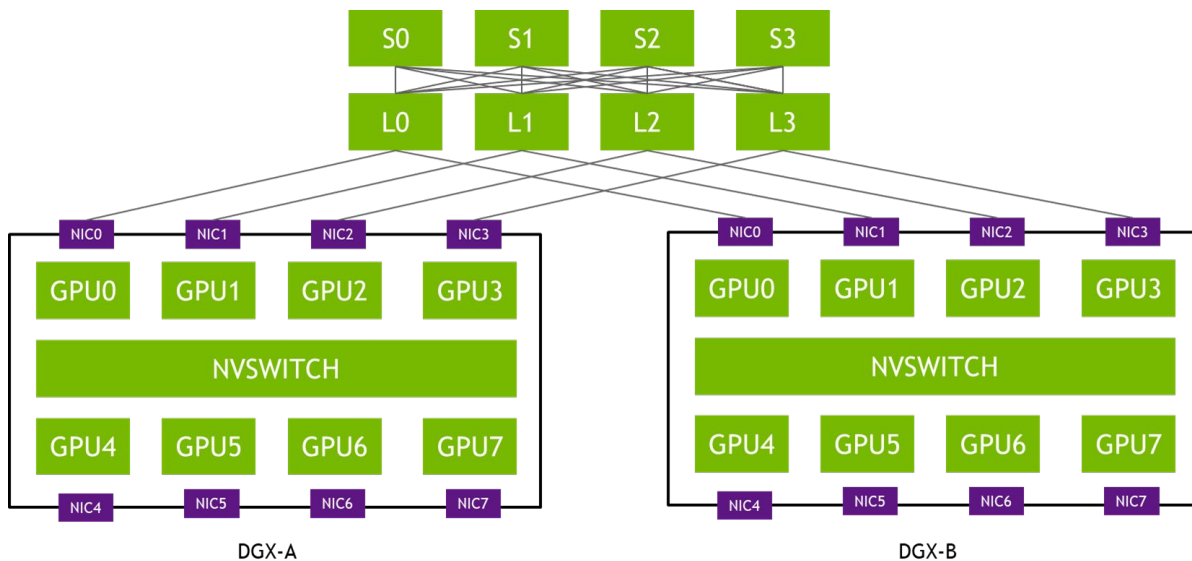


From HPC to ML Systems: A System View



Network Communication in NVIDIA

- Inter-node: S -> Spine; L -> Leaf; NIC -> network interface card.
- Intra-node: NVSwitch, NVLink, PCI-e.



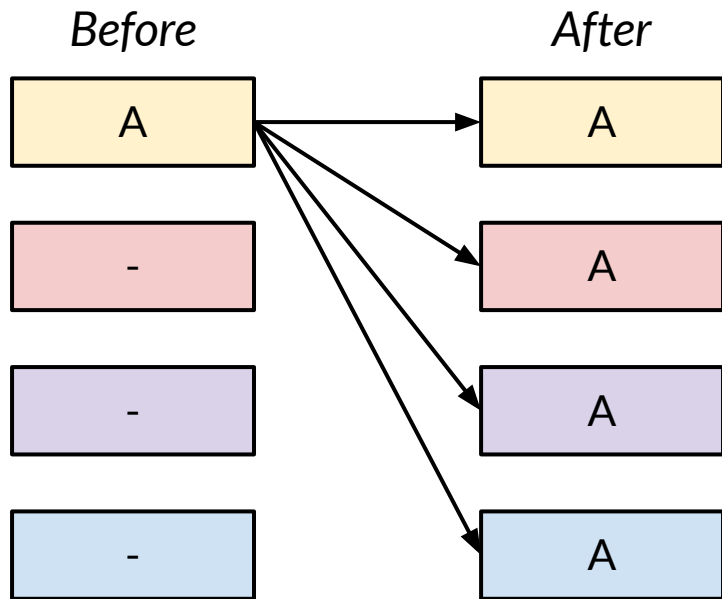
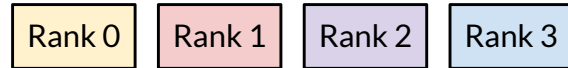
NVIDIA GPU Cluster Network Topology^[1]

[1] <https://developer.nvidia.com/blog/doubling-all2all-performance-with-nvidia-collective-communication-library-2-12/>

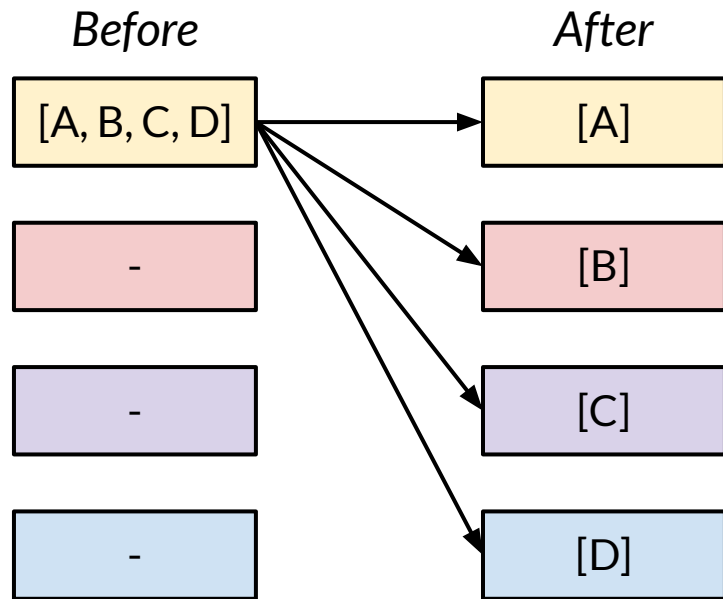
Communication Vocabulary

- Process: an independent running instance of a program.
 - Rank: the unique ID number assigned to the process in a communication group.
 - Message/buffer: data that is communicated across each rank.
 - Point-to-point: communication between a pair of process.
 - Collectives: communication involve a group of processes.
-
- Communication libraries: MPI, NCCL, RCCL, PyTorch Distributed, etc.

Collective Operation: One-to-Many

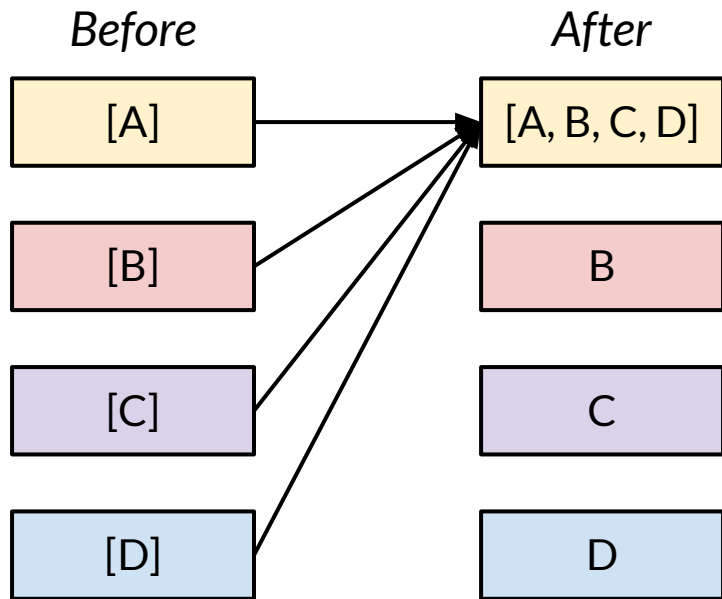
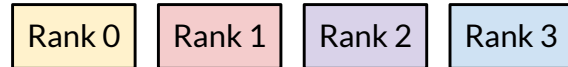


Broadcast

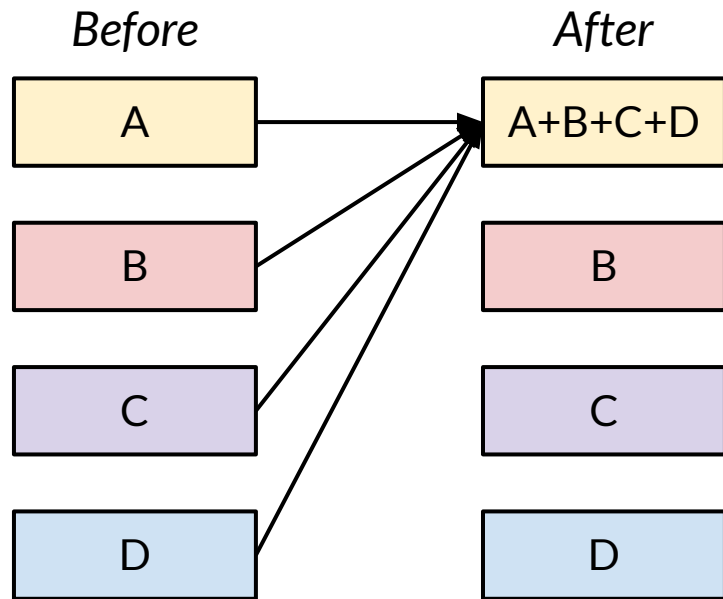


Scatter

Collective Operation: Many-to-One

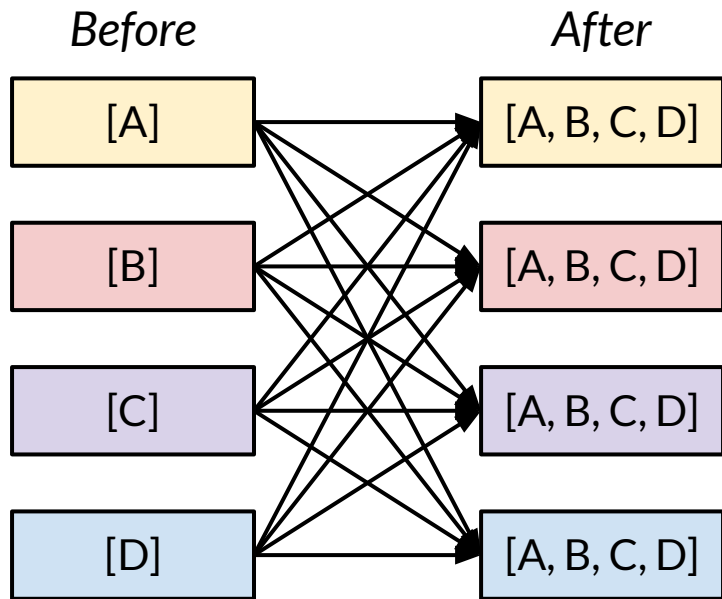
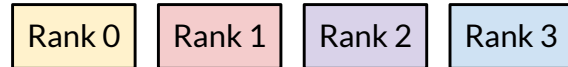


Gather

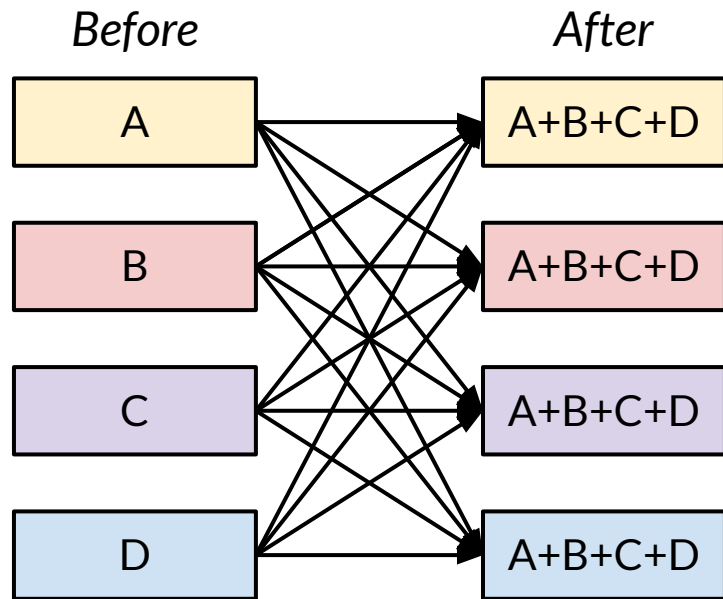


Reduce

Collective Operation: Many-to-Many

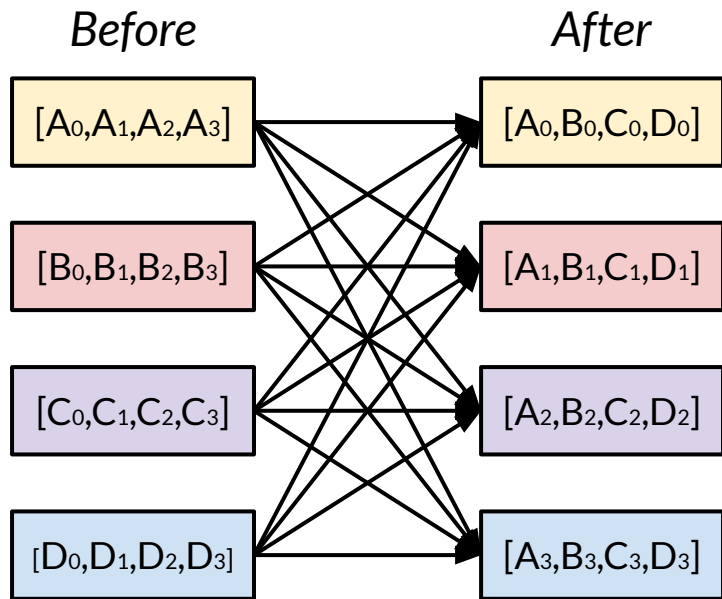


AllGather

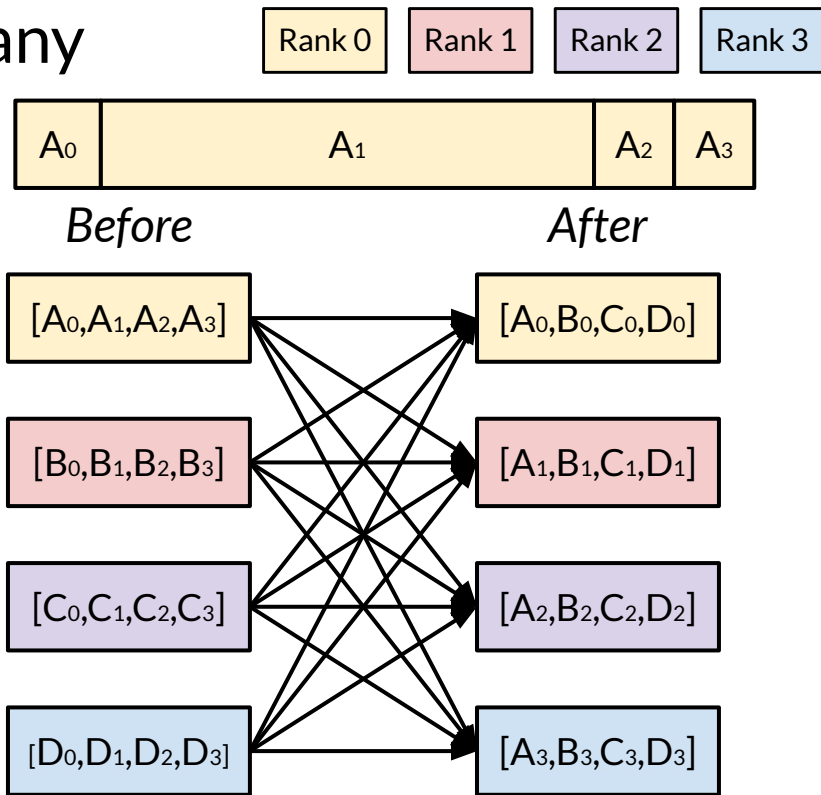


AllReduce

Collective Operation: Many-to-Many



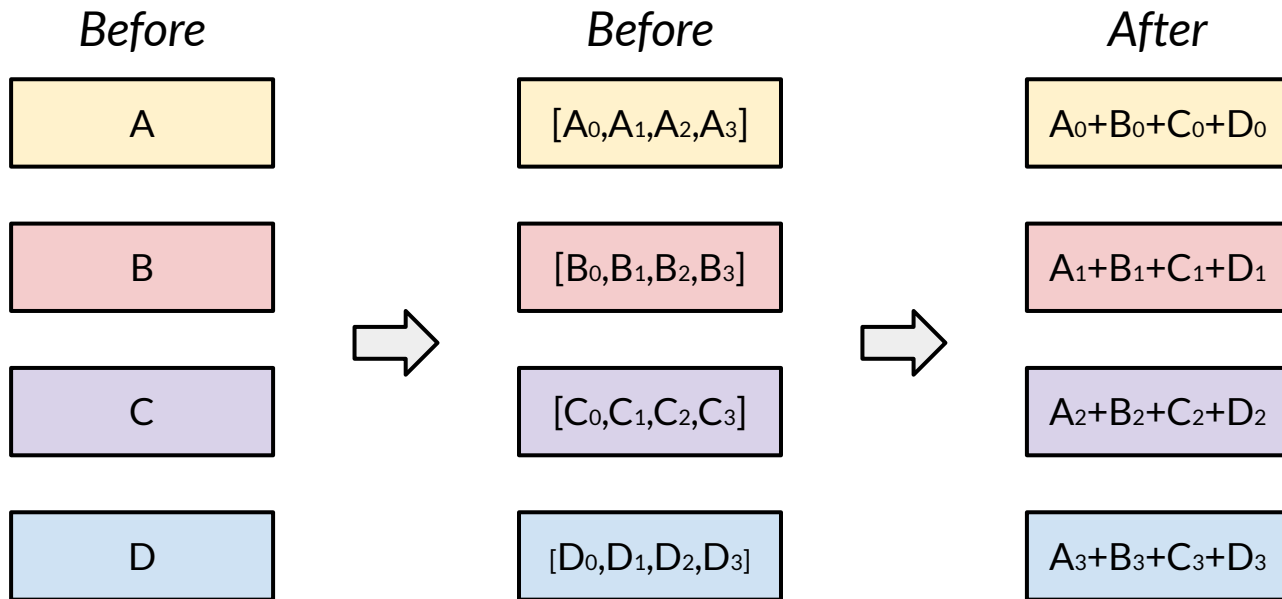
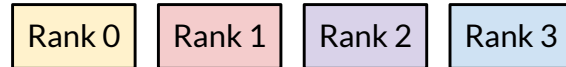
All-to-All



All-to-Allv

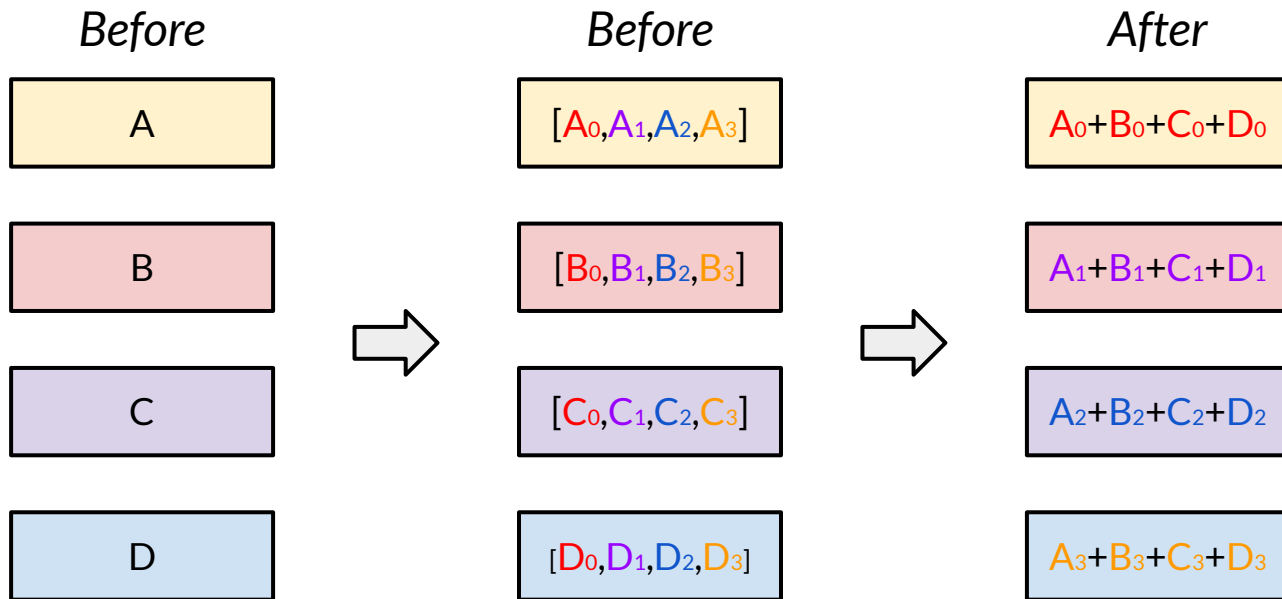
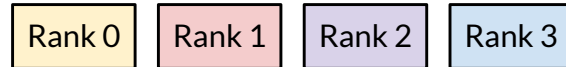
variable size/counts

Collective Operation: Specialized



ReduceScatter

Collective Operation: Specialized



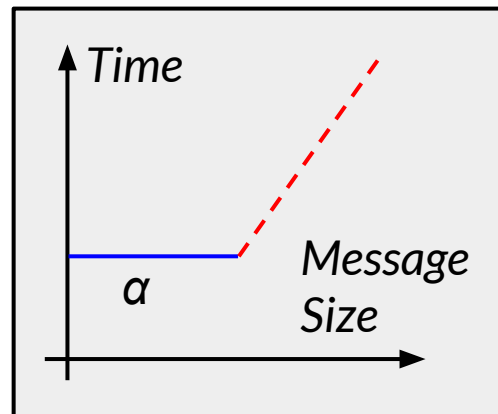
ReduceScatter

Collective Operation $\neq$ Collective Algorithm

- Operation / Primitive: specifies **what** data each rank should receive.
- Algorithm specifies **how** data moves among ranks.
- AllReduce algorithm:
 - Centralized
 - Ring
 - Recursive doubling

Communication Cost Model

- Modeling communication cost: $T_{\text{msg}}(n) = \alpha + \beta n$
 - α = latency / startup cost
 - β = Time per byte ($\approx$ reciprocal of bandwidth)
 - n = message size
- Small messages -> latency-bound.
 - Minimize number of communication rounds.
- Large messages -> bandwidth-bound.
 - Minimize the total communicated bytes.
 - Maximize bandwidth utilization.



Collective Cost Generalization

- Modeling communication cost: $T_{\text{msg}}(n) = \alpha + \beta n$
- Collective communication cost: $T_{\text{collective}} \approx N_{\text{steps}}\alpha + V_{\text{comm}}\beta$
 - N_{steps} = communication rounds on critical path.
 - V_{comm} = bytes communicated by a rank.
- What makes a good collective algorithm?
 - Few steps.
 - Low communication volume.
 - High memory bandwidth utilization.